# Earned Autonomy: A Reversibility and Evidence Gate for Automated Infrastructure Remediation

Aman Das
Earned Autonomy Project
Remote, India
Supervisor: Kother Badushah

***Abstract*—Automated remediation can reduce recovery time and operator workload, but an incorrect automatic action can turn an existing incident into a larger one. This paper presents EarnedAutonomy, a small trust gate positioned between a fault detector and a remediation executor. The gate returns AUTO, PROPOSE, or ESCALATE using three factors required by the project design: reversibility, blast radius, and accumulated evidence. Evidence is maintained per action type and machine class using a Beta posterior, with a conservative lower posterior percentile used instead of the posterior mean. The implementation uses a public Blue Gene/L (BGL) system-log dataset only to inform the simulated incident distribution; remediation ground truth is provided by a controlled simulator. The simulator models six remediation actions, resolved, recurrent, and harmful outcomes, and feedback into the evidence store. Evaluation compares EarnedAutonomy with Always Ask and Always Act using automatic resolution rate, mean time to recovery, human interruptions, and unsafe automatic executions. A deterministic seed-42 benchmark gives EarnedAutonomy an automation rate of 0.2% and two unsafe automatic executions, compared with 100% automation and 472 unsafe executions for Always Act. A second seed produces 1.1% automation and zero unsafe automatic executions for EarnedAutonomy. Threshold sweeps show the expected trade-off: more permissive automatic-execution thresholds increase autonomy while increasing unsafe automatic actions in the simulated setting. The results demonstrate a conservative earned-autonomy mechanism rather than a claim of universal dominance over simpler policies.**

***Index Terms*—automated remediation, self-healing infrastructure, Bayesian evidence, trust gate, self-adaptive systems, infrastructure reliability**

## I. Introduction

Large-scale computing environments increasingly depend on automation to monitor systems, detect faults, and initiate recovery actions. The broader research area of autonomic and self-adaptive computing has long considered self-healing as a mechanism for maintaining system objectives despite faults and changing operating conditions [1], [2], [4], [5]. However, automation of the action itself introduces a different question: when should a system be permitted to act without asking a human?

A fixed allow list can answer this question operationally, but it does not naturally improve as evidence accumulates. An action may be safe in one context and unreliable in another, and the cost of an incorrect action can vary substantially with its scope. The project therefore treats autonomy as a permission that must be earned rather than as a binary property of the remediation system.

EarnedAutonomy is deliberately narrower than a complete self-healing system. It does not detect failures, diagnose incidents, execute real infrastructure actions, or build a monitoring interface. Instead, it accepts a candidate remediation and determines whether that action should be AUTO, PROPOSE, or ESCALATE. The surrounding simulator supplies the incident and the observed outcome, allowing the gate to learn from repeated remediation attempts.

The contribution is a small and interpretable trust gate that combines three simple quantities. Reversibility captures how easily an action can be undone. Blast radius captures how much infrastructure may be affected if the action is wrong. Evidence captures how well the action has performed previously in the same machine-class context. The evidence component is intentionally conservative: a lower percentile of a Beta posterior is used so that a small number of successful observations does not immediately justify autonomous execution.

The evaluation emphasizes unsafe automatic executions because the project is intended for settings where an incorrect automatic action can be more costly than a slower human-approved action. The resulting experiments therefore report both automation and safety rather than optimizing a single performance metric.

## II. Problem Statement

A conventional remediation pipeline can be written as

$$\text{Fault Detection} \rightarrow \text{Candidate Action} \rightarrow \text{Execution}. \tag{1}$$

EarnedAutonomy inserts a decision layer:

$$\begin{aligned}\text{Fault Detection} \rightarrow \text{Candidate Action} \rightarrow \text{Trust Gate} \\ \rightarrow \{\text{AUTO}, \text{PROPOSE}, \text{ESCALATE}\}.\end{aligned} \tag{2}$$

The design must satisfy two competing requirements. First, an action with no relevant history should not immediately receive permission to execute automatically. Second, an action with a large potential impact should remain restricted even if its observed success rate is high. The gate must therefore combine historical evidence with explicit operational risk.

The context in which an action is used is also important. Tracking evidence only by action type can transfer experience between incompatible machine classes. Tracking by individual

machine can make the evidence too sparse. The chosen middle ground is a small set of machine classes such as compute node, storage node, and scheduler, with evidence stored for each action–class pair.

## III. Related Work

Autonomic computing established the broader vision of systems that manage themselves through capabilities including self-configuration, self-optimization, self-healing, and self-protection [1]. Surveys of self-healing systems subsequently examined recovery mechanisms, decision support, and fault-tolerant adaptation [2], [3]. Self-adaptive software extends this perspective by treating runtime decision making as a response to changes in system or environmental context [4], [5]. A recent systematic review similarly characterizes self-adaptive systems in terms of context awareness, runtime adaptation, and the balance between closed-loop automation and human involvement [6].

The present work is related to these feedback-oriented approaches but focuses on a narrower control question: permission to execute a remediation action. Instead of learning a failure detector or selecting an optimal repair plan, the gate assumes that an upstream component has already produced a candidate action. The gate then evaluates whether that action has earned sufficient trust to cross an autonomy boundary.

Operational log analysis provides another foundation for the project. Oliner and Stearley showed the value of studying real system logs from large computing environments and highlighted the difficulty of translating log observations into useful failure information [7]. The BGL dataset used here follows this motivation: its purpose is to provide realistic incident-frequency information, not remediation ground truth.

The evidence model uses Bayesian updating rather than a raw empirical success rate. Beta-binomial updating provides a compact representation of uncertainty for repeated binary outcomes, and the use of a lower posterior percentile makes the decision conservative when evidence is sparse. The implementation uses SciPy's established statistical routines rather than implementing a numerical inverse CDF directly [10].

Recent work also highlights the validation and safety challenges of increasingly autonomous self-healing systems. Systematic reviews identify the need for controlled evaluation and explicit treatment of human-in-the-loop safety when autonomous repair decisions are deployed in uncertain environments [11]. EarnedAutonomy follows this principle by making unsafe automatic execution a first-class benchmark metric and by keeping the simulator assumptions explicit.

## IV. Design

### A. System Flow

The system follows five steps. First, an upstream incident source provides an incident record. Second, a lookup-based classifier maps the incident to one candidate remediation action. Third, EarnedAutonomy evaluates the action in a machine-class context. Fourth, the simulated environment produces an outcome. Finally, the outcome updates the evidence store and affects later decisions.

### B. The Three Factors

**Reversibility** is a value in $[0, 1]$ representing how easily an incorrect action can be undone. Restarting a stateless service receives a high value, whereas reclaiming disk space by deleting temporary files receives a lower value.

**Blast radius** is normalized to $[0, 1]$ and represents the potential scope of impact. Larger values push the decision toward ESCALATE regardless of historical evidence.

**Evidence** measures observed remediation performance in a particular context. Evidence is stored for $(\text{action type}, \text{machine class})$ pairs rather than for actions alone.

### C. Bayesian Evidence

For every action–machine-class pair, the prior is

$$p \sim \mathrm{Beta}(1, 1). \tag{3}$$

A resolved outcome increments $\alpha$ and a recurrent or harmful outcome increments $\beta$:

$$\alpha \leftarrow \alpha + 1, \qquad \beta \leftarrow \beta + 1. \tag{4}$$

The posterior mean is

$$E[p] = \frac{\alpha}{\alpha + \beta}, \tag{5}$$

but it is not used by the gate. Instead, the evidence score is the fifth percentile of the posterior:

$$E_{LB} = F^{-1}_{\mathrm{Beta}(\alpha,\beta)}(0.05). \tag{6}$$

This makes two successful observations less persuasive than a large body of successful observations even when both empirical success rates are 100%.

### D. Decision Rule

The gate calculates

$$Trust = Reversibility \times E_{LB} \tag{7}$$

and

$$Risk = Normalized\ Blast\ Radius. \tag{8}$$

The configured decision rule is

$$\begin{cases} AUTO, & Trust > T_A \wedge Risk < R_C \wedge History, \\ PROPOSE, & Trust > T_P, \\ ESCALATE, & \text{otherwise.} \end{cases} \tag{9}$$

The additional history requirement prevents the Beta prior itself from being treated as sufficient evidence for autonomous execution.

TABLE I
TRUST GATE CONFIGURATION

| Parameter | Value |
|---|---|
| AUTO threshold | 0.65 |
| PROPOSE threshold | 0.35 |
| Risk ceiling | 0.60 |
| Evidence percentile | 0.05 |

### E. Configuration

The current configuration is:

Action factors are stored separately in YAML. The six required actions and configured values are shown in Table II.

TABLE II
REMEDIATION ACTIONS

| Action | Reversibility | Blast Radius |
|---|---|---|
| Restart service | 0.9 | 0.1 |
| Requeue job | 0.9 | 0.1 |
| Reset driver | 0.6 | 0.3 |
| Drain node | 0.5 | 0.7 |
| Reclaim disk | 0.3 | 0.5 |
| Escalate | 1.0 | 0.0 |

### F. Audit Logging

Every simulated gate decision is represented as a structured record containing the timestamp, incident identifier and type, candidate action, machine class, reversibility, blast radius, evidence lower bound, trust, risk, decision, outcome, resolution status, and recovery time. This provides the audit trail needed to reconstruct benchmark decisions rather than relying only on aggregate metrics.

## V. DATA AND SIMULATOR

### A. BGL Dataset

The Blue Gene/L system-log dataset was selected because it provides labelled system events from a large-scale computing environment [7], [8]. The local analysis contains 4,747,963 records, including 4,399,503 normal records and 348,460 alert records. The alert rate is 7.34%, and 42 distinct labels are present.

The most frequent alert labels include KERNDTLB (152,734), KERNSTOR (63,491), APPSEV (49,651), KERNMNTF (31,531), and KERNTERM (23,338). These counts are used only to derive a simplified incident distribution. The BGL dataset does not provide ground-truth remediation actions for the benchmark.

To satisfy the required log-preprocessing step, the repository also applies the Drain log-template extraction approach to the unstructured message portion of BGL records [9]. Drain uses a fixed-depth parse tree to narrow candidate log groups and then generalizes differing token positions into template wildcards. This project uses a small deterministic implementation of that core workflow in `src/log_templates.py`. Template mining is performed on a 10,000-record sample by default, while the label-frequency calculation scans the complete BGL file. Thus, template extraction is used to group similar log lines without turning the project into a full log parser.

### B. Incident Taxonomy

The simulator uses five broad incident categories:

- kernel error;
- storage error;
- filesystem failure;
- communication failure;
- application error.

Three machine classes are used: compute node, storage node, and scheduler. Each incident contains an identifier, incident type, machine class, and simulated timestamp. The simulator generates sequential identifiers and can produce 1,000 incidents in one run.

### C. Incident Distribution

The incident weights used by the simulator are:

TABLE III
SIMULATOR INCIDENT WEIGHTS

| Incident type | Weight |
|---|---|
| Kernel error | 0.5235 |
| Storage error | 0.1831 |
| Filesystem failure | 0.0930 |
| Communication failure | 0.0177 |
| Application error | 0.1827 |

These are simulation assumptions informed by the observed alert distribution. They are not claimed to be exact mappings from BGL labels to remediation scenarios.

Figure 1 shows the generated distribution for the 1,000-incident run.

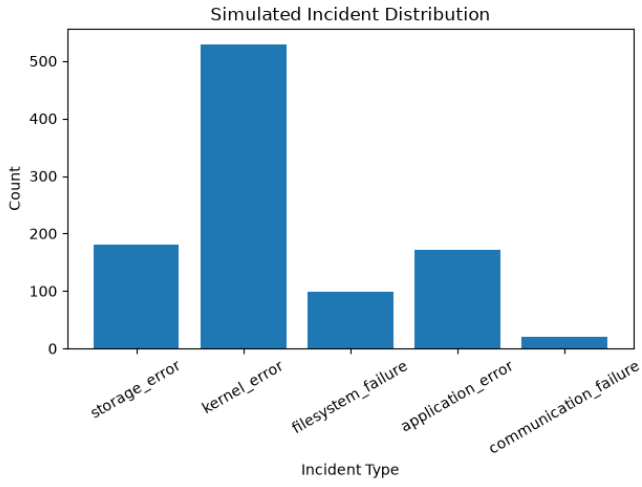



Fig. 1. Incident distribution generated by the simulator for the 1,000-incident run.

### D. Outcome Model

The simulator distinguishes three outcomes. RESOLVED indicates successful remediation, RECURRED represents an action that appears to work but does not resolve the underlying cause, and HARMFUL represents a remediation that makes the simulated incident worse.

The fixed action-conditioned probabilities are:

TABLE IV
SIMULATOR OUTCOME PROBABILITIES

| Action | Resolved | Recurred | Harmful |
|---|---|---|---|
| Restart service | 0.80 | 0.15 | 0.05 |
| Requeue job | 0.45 | 0.35 | 0.20 |
| Reset driver | 0.60 | 0.25 | 0.15 |
| Drain node | 0.50 | 0.25 | 0.25 |
| Reclaim disk | 0.40 | 0.30 | 0.30 |

These probabilities are fixed simulator assumptions. They are not learned from BGL and are not tuned during the benchmark to make EarnedAutonomy win.

## VI. Implementation and Verification

The core implementation is intentionally small. The gate is implemented in `src/gate.py`, the Beta evidence model in `src/evidence.py`, and incident and outcome simulation in `src/simulator.py`. Baseline policies are in `src/baselines.py`; `benchmark.py` executes the controlled comparison; and `threshold_sweep.py` performs sensitivity analysis.

The final test suite contains 13 tests. It covers the no-history AUTO safety condition, the high-blast-radius condition, PROPOSE and AUTO behavior, baseline policy behavior, metric edge cases, benchmark determinism, unsafe-execution counting, and the presence of the required audit fields in decision records. The final run reported 13 passing tests.

The gate was also run against 1,000 incidents with no prior evidence. In that state the lower evidence bound is insufficient for automatic execution, so the initial gate verification produces zero AUTO decisions and keeps the decisions under human control.

## VII. Evidence Accumulation Experiment

The learning experiment evaluates two known action contexts using 100 simulated incidents per action and seed 42. The first context is restart service for a kernel error on a compute node. The second is drain node for a communication failure on a compute node.

The restart-service action begins without history and therefore starts at ESCALATE. As outcomes are observed, successful observations increase the Beta posterior and can move the decision through PROPOSE to AUTO. In the current run, the first AUTO decision occurs at incident 16. Across the 100 simulated incidents the decision counts are 3 ESCALATE, 50 PROPOSE, and 47 AUTO.

The drain-node action has a configured blast radius of 0.7, above the 0.60 risk ceiling. It therefore remains excluded from AUTO even when evidence is accumulated. Its 100 decisions are all ESCALATE.

Figure 2 plots the running proportion of AUTO decisions. The plotted value is cumulative: it represents the fraction of observed decisions classified as AUTO up to each incident.

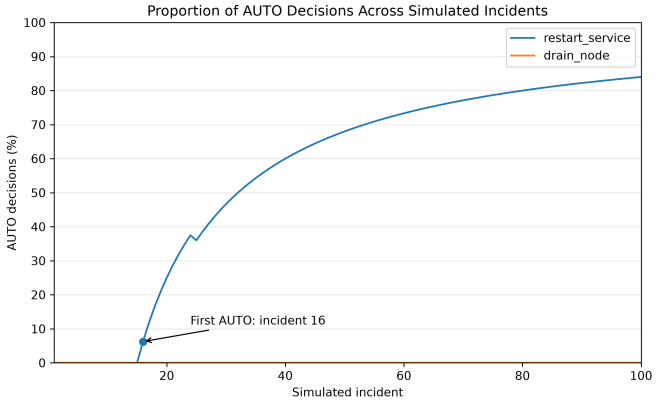



Fig. 2. Running proportion of AUTO decisions during evidence accumulation. Restart service earns AUTO while drain node remains excluded by the risk ceiling.

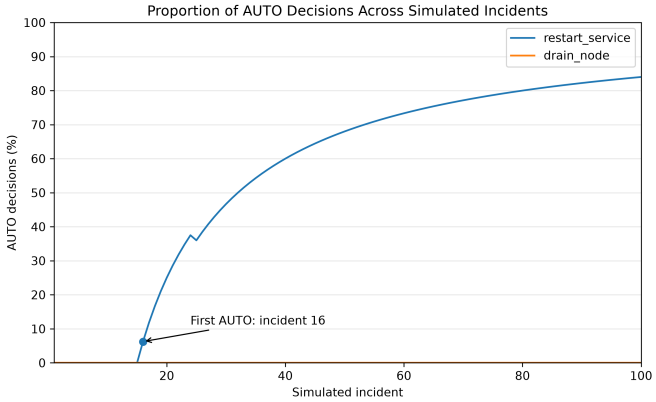



Fig. 3. Week 3 AUTO proportion.

This experiment satisfies the central earned-autonomy behavior required by the design: one action progresses from ESCALATE to AUTO, while another never becomes AUTO because of its risk profile.

## VIII. Experimental Methodology

The benchmark compares three policies over the same deterministic incident stream:

1) **Always Ask**: every candidate action receives human approval.
2) **Always Act**: every candidate action is executed automatically.
3) **EarnedAutonomy**: every candidate action passes through the trust gate.

The four required metrics are defined as follows. Automatic resolution rate is the proportion of all incidents whose action

was both AUTO and resolved. Mean time to recovery is the average simulated time from incident to final resolution. Human interruptions count incidents requiring human involvement before execution or during recovery. Unsafe automatic executions count AUTO decisions followed by RECURRED or HARMFUL outcomes.

The benchmark models human approval as a fixed 120-second delay and human recovery as a fixed 300-second delay. Action execution times are configured per remediation type. These values are explicit simulation assumptions and are held fixed during threshold sweeps.

The incident-to-action mapping is a lookup table. This is an explicit benchmark assumption because BGL provides log observations rather than remediation labels. The benchmark therefore tests the gate under a controlled ground-truth simulation rather than claiming production remediation accuracy.

## IX. Benchmark Results

### A. Seed 42

Table V gives the benchmark results for the default deterministic seed of 42.

TABLE V
BENCHMARK RESULTS FOR SEED 42

| Policy | Auto Res. | MTTR (s) | Human | Unsafe Auto |
|---|---|---|---|---|
| Always Ask | 0.000 | 291.685 | 1000 | 0 |
| Always Act | 0.528 | 171.685 | 472 | 472 |
| EarnedAutonomy | 0.000 | 443.905 | 1000 | 2 |

Always Act provides the highest automation and the lowest mean recovery time in this run, but it also produces 472 unsafe automatic executions. EarnedAutonomy allows only two autonomous executions, both unsafe in this particular seed. This is a deliberately important negative result: the gate is conservative, but its current threshold does not guarantee zero unsafe executions.

Always Ask produces no unsafe automatic executions because it never executes an action autonomously, but it also produces no automatic resolutions and requires human involvement for all 1,000 incidents.

### B. Seed 43

A second deterministic run uses seed 43. The results are shown in Table VI.

TABLE VI
BENCHMARK RESULTS FOR SEED 43

| Policy | Auto Res. | MTTR (s) | Human | Unsafe Auto |
|---|---|---|---|---|
| Always Ask | 0.000 | 288.300 | 1000 | 0 |
| Always Act | 0.538 | 168.300 | 462 | 462 |
| EarnedAutonomy | 0.011 | 442.320 | 989 | 0 |

For seed 43, EarnedAutonomy performs 11 autonomous executions, achieves an automatic resolution rate of 0.011, and records zero unsafe automatic executions. Always Act again reaches 100% automation but produces 462 unsafe automatic executions.

The second seed therefore does not reverse the qualitative conclusion. The gate remains much less autonomous than Always Act, while producing dramatically fewer unsafe automatic executions in absolute terms.

## X. Threshold Sensitivity

The threshold sweep evaluates AUTO thresholds from 0.50 to 0.95 and PROPOSE thresholds from 0.20 to 0.50. Configurations where PROPOSE is greater than or equal to AUTO are excluded, leaving 69 valid threshold combinations.

At PROPOSE=0.35 for seed 42, the selected AUTO-threshold results are shown in Table VII.

TABLE VII
SELECTED SEED-42 THRESHOLD SWEEP RESULTS AT PROPOSE = 0.35

| AUTO | Automation | Unsafe Auto |
|---|---|---|
| 0.50 | 3.7% | 9 |
| 0.55 | 2.5% | 7 |
| 0.60 | 1.1% | 4 |
| 0.65 | 0.2% | 2 |
| 0.70 | 0.0% | 0 |
| 0.75 | 0.0% | 0 |
| 0.80 | 0.0% | 0 |
| 0.85 | 0.0% | 0 |
| 0.90 | 0.0% | 0 |
| 0.95 | 0.0% | 0 |

The threshold sweep shows the expected safety–automation trade-off. More permissive AUTO thresholds increase autonomous execution, but they also increase unsafe automatic executions in the seed-42 simulation. At thresholds of 0.70 and above, no autonomous execution occurs in this run.

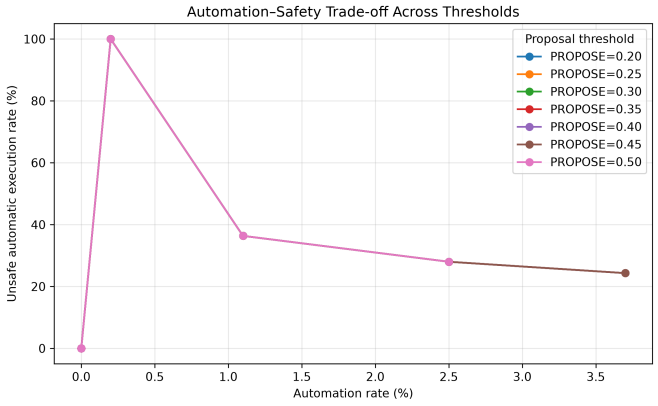



Fig. 4. Automation–safety trade-off across threshold configurations for seed 42. The x-axis is automation rate and the y-axis is unsafe automatic execution rate.

The same sweep was executed with seed 43. At PROPOSE=0.35, the AUTO thresholds 0.50, 0.55, 0.60, 0.65, and 0.70 produce automation rates of 3.9%, 2.8%, 2.0%, 1.1%, and 0.7%, respectively. The corresponding unsafe automatic execution counts are 5, 3, 2, 0, and 0. This second sweep preserves

the same qualitative relationship between permissiveness and safety.

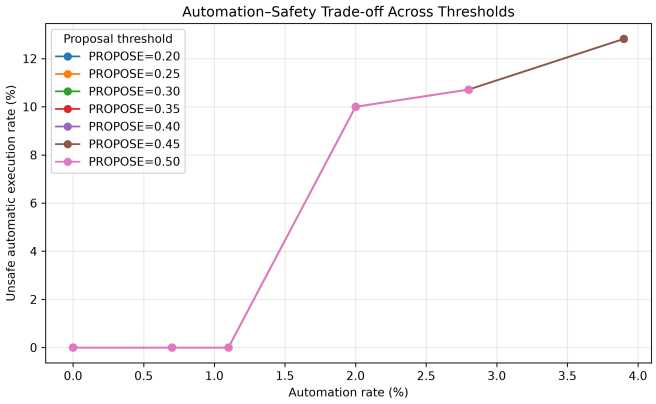



Fig. 5. Automation–safety trade-off across threshold configurations for seed 43.

## XI. Conditions Where EarnedAutonomy Does Not Dominate

The experiments identify multiple conditions where EarnedAutonomy performs no better than a baseline.

First, when the objective is minimum mean time to recovery and unsafe automation is ignored, Always Act is better. For seed 42, Always Act reaches 171.685 seconds compared with 443.905 seconds for EarnedAutonomy. For seed 43, the values are 168.300 and 442.320 seconds, respectively.

Second, when the objective is maximum automatic resolution rate, Always Act is also better. Its automatic resolution rates are 0.528 and 0.538 for seeds 42 and 43, while EarnedAutonomy produces 0.000 and 0.011.

Third, at the default threshold in seed 42, EarnedAutonomy's two autonomous executions are both unsafe. This shows that the gate's conservative automation rate is not by itself sufficient to guarantee safe autonomy.

These limitations are useful results rather than failures to hide. The project is designed to expose the trade-off between autonomy and safety, not to force the proposed gate to outperform every baseline on every metric.

## XII. Limitations and Threats to Validity

The simulator is not a production infrastructure environment. Its action outcome probabilities, action execution times, human delays, reversibility values, and blast-radius values are modeling assumptions. They should not be interpreted as production measurements.

The BGL dataset informs incident-frequency shape but does not provide remediation ground truth. The benchmark therefore relies on a deterministic incident-to-action lookup table. This provides controlled ground truth for unsafe-execution measurement but limits external validity.

The evidence model treats resolved outcomes as successes and both recurrent and harmful outcomes as failures. This is deliberately simple. Real systems could distinguish partial recovery, repeated failure severity, or action-specific costs, but adding those factors would exceed the project's defined scope.

The learning experiment allows supervised handling after ESCALATE or PROPOSE to produce an outcome that contributes evidence. This models the practical idea that human-approved remediation can still generate experience for future autonomous decisions; it is not an assertion that the gate executed those actions autonomously.

The current results use two deterministic seeds. They establish reproducible behavior and a consistent qualitative trade-off, but they do not constitute a statistical estimate of production performance.

## XIII. Safety Properties and Design Trade-offs

The gate has two explicit safety properties that are tested directly. First, an action with no observed history cannot receive AUTO. The initial Beta prior therefore does not grant automatic permission by itself. Second, an action whose normalized blast radius reaches the configured risk ceiling cannot receive AUTO, even after successful observations. The tests exercise both properties independently.

### A. No-History Constraint

The no-history constraint addresses the uncertainty represented by the Beta prior. With a Beta(1,1) prior, the posterior mean is 0.5, but that value does not represent observed remediation performance. Using the lower fifth percentile and additionally requiring observed history prevents the system from interpreting the prior as empirical evidence. This makes the initial behavior conservative and forces an action to accumulate observations before it can cross the AUTO boundary.

### B. Blast-Radius Constraint

The risk ceiling is an independent control from evidence. Drain-node has a configured blast radius of 0.7 while the risk ceiling is 0.60. Consequently, even a sequence of successful drain-node outcomes cannot satisfy the AUTO condition. This separation is important because evidence answers whether an action has worked before, whereas blast radius describes how costly an incorrect action could be.

### C. Context-Specific Evidence

Evidence is indexed by both action type and machine class. The design uses three classes rather than individual machine identifiers so that the evidence pool remains large enough to learn while retaining operational context. For example, evidence for restart service on compute nodes is not automatically used for restart service on storage nodes. This is a deliberate compromise between over-generalization and excessively sparse evidence.

### D. Why the Gate Can Be Worse Than a Baseline

The conservative design necessarily creates situations where EarnedAutonomy is slower than Always Act. A PROPOSE or ESCALATE decision adds human delay even when the underlying action would have succeeded. The current benchmark makes this cost explicit through fixed human approval

and recovery delays. This is not an implementation defect hidden by the evaluation; it is the operational price of reducing autonomous exposure.

The converse is also visible in the benchmark. Always Act has no mechanism to use accumulated evidence, action reversibility, or blast radius to restrict execution. It therefore achieves maximum automation but exposes every incident to autonomous action. The comparison makes the project's intended design trade-off measurable rather than assuming that either extreme is universally correct.

## XIV. Detailed Benchmark Interpretation

The four primary metrics are intentionally complementary. Automation rate measures how often the gate permits autonomous execution, whereas automatic resolution rate asks whether those autonomous executions actually resolve incidents. Mean time to recovery captures the time cost of both automation and human intervention. Human interruptions measures the operational burden placed on people. Unsafe automatic executions directly measure the failure mode that the trust gate is intended to control.

Table VIII reports the additional rates emitted by the benchmark. These values are not used as replacements for the four required metrics; they provide context for interpreting the primary results.

TABLE VIII
Additional Benchmark Metrics

| Metric | Always Ask | Always Act | EarnedAutonomy |
|---|---|---|---|
| Seed 42 | | | |
| Automation rate | 0.000 | 1.000 | 0.002 |
| Unsafe auto rate | 0.000 | 0.472 | 1.000 |
| Remediation success | 0.528 | 0.528 | 0.528 |
| Seed 43 | | | |
| Automation rate | 0.000 | 1.000 | 0.011 |
| Unsafe auto rate | 0.000 | 0.462 | 0.000 |
| Remediation success | 0.538 | 0.538 | 0.538 |

The remediation success rate is equal across policies for a given seed because the same simulated incident stream and action-conditioned outcome model are used for the comparison. The policy changes the level of human involvement and whether an outcome is classified as an unsafe autonomous execution; it does not change the underlying simulator probabilities.

The seed-42 unsafe automatic execution rate for EarnedAutonomy is 1.000 even though its automation rate is only 0.002. This distinction is important. A small number of autonomous executions does not automatically imply that those executions are safe. Conversely, seed 43 demonstrates that the same gate can produce autonomous executions without an unsafe execution under a different deterministic incident sequence.

### A. Decision Record Schema

The benchmark stores structured records rather than only aggregate counts. Table IX summarizes the fields written for each simulated decision.

TABLE IX
Decision Record Fields

| Field | Purpose |
|---|---|
| Timestamp | Time associated with the simulated incident |
| Incident ID/type | Identifies the incident and its category |
| Candidate action | Remediation selected by the lookup classifier |
| Machine class | Context used for evidence isolation |
| Reversibility | Action reversibility at decision time |
| Blast radius | Normalized action impact at decision time |
| Evidence lower bound | Conservative Beta-posterior evidence value |
| Trust and risk | Values used by the gate rule |
| Decision | AUTO, PROPOSE, or ESCALATE |
| Outcome/resolution | Observed result and resolution state |
| Recovery time | Simulated time from incident to final resolution |

This schema allows a reviewer to trace an aggregate benchmark metric back to the underlying simulated decisions. It also makes the distinction between the policy decision and the eventual outcome explicit.

## XV. Reproducibility

The repository uses Python 3.10 or newer and requires pytest, matplotlib, SciPy, and PyYAML. The core gate remains a small set of Python modules without a heavy framework.

The benchmark is run with:

```
python benchmark.py
```

A second seed is run with:

```
python benchmark.py --seed 43
```

Threshold sensitivity is reproduced with:

```
python threshold_sweep.py --seed 42
python threshold_sweep.py --seed 43
```

The evidence accumulation experiment is reproduced with:

```
python run_learning_experiment.py
```

The test suite is run with:

```
python -m pytest -q
```

The final verification reported 13 passing tests. Benchmark outputs are stored as seed-specific JSON files, and the default seed-42 command also writes the repository-level benchmark result and summary files. The four paper figures are stored in `figures/`.

## XVI. Conclusion and Future Work

EarnedAutonomy implements a small trust gate that makes autonomous remediation conditional on reversibility, blast radius, and accumulated context-specific evidence. The Beta posterior lower bound provides a conservative evidence score, while the explicit risk ceiling prevents high-blast-radius actions from becoming AUTO solely because of historical success.

The controlled learning experiment demonstrates earned autonomy: restart service progresses from ESCALATE through PROPOSE to AUTO, while drain node remains excluded from AUTO. The benchmark demonstrates the central trade-off. Always Act is faster and more autonomous but generates hundreds of unsafe automatic executions. EarnedAutonomy is substantially less autonomous and slower, but its unsafe automatic execution count is much lower in absolute terms across the two evaluated seeds.

The threshold sweep further shows that autonomy cannot be increased for free. Lower AUTO thresholds increase autonomous execution and, in the seed-42 run, increase unsafe automatic executions. The results therefore support a safety-first interpretation of earned autonomy rather than an automation-first one.

Future work within a production setting would require calibrated risk values, production remediation outcomes, and broader evaluation across infrastructure contexts. Those extensions are outside the present simulator-based scope. The current artifact instead provides a small, reproducible, and inspectable gate and benchmark that makes its assumptions and limitations explicit.

## REFERENCES


[1] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003, doi: 10.1109/MC.2003.1160055.

[2] M. C. Huebscher and J. A. McCann, "A survey of autonomic computing—degrees, models, and applications," *ACM Computing Surveys*, vol. 40, no. 3, 2008, doi: 10.1145/1380584.1380585.

[3] D. Ghosh, R. Sharman, H. Raghav Rao, and S. Upadhyaya, "Self-healing systems—survey and synthesis," *Decision Support Systems*, vol. 42, no. 4, pp. 2164–2185, 2007, doi: 10.1016/j.dss.2006.06.011.

[4] F. D. Macías-Escrivá, R. Haber, R. del Toro, and V. Hernández, "Self-adaptive systems: A survey of current approaches, research challenges and applications," *Expert Systems with Applications*, vol. 40, no. 18, pp. 7267–7279, 2013, doi: 10.1016/j.eswa.2013.07.033.

[5] C. Krupitzer, F. M. Roth, S. VanSyckel, G. Schiele, and C. Becker, "A survey on engineering approaches for self-adaptive systems," *Pervasive and Mobile Computing*, vol. 17, pp. 184–206, 2015, doi: 10.1016/j.pmcj.2014.09.009.

[6] T. Wong, M. Wagner, and C. Treude, "Self-adaptive systems: A systematic literature review across categories and domains," *Information and Software Technology*, vol. 148, 106934, 2022, doi: 10.1016/j.infsof.2022.106934.

[7] A. J. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *Proc. IEEE/IFIP International Conference on Dependable Systems and Networks*, pp. 575–584, 2007, doi: 10.1109/DSN.2007.103.

[8] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in *Proc. IEEE International Symposium on Software Reliability Engineering*, pp. 355–366, 2023, doi: 10.1109/ISSRE59848.2023.00071.

[9] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An Online Log Parsing Approach with Fixed Depth Tree," in *Proc. IEEE International Conference on Web Services*, pp. 33–40, 2017, doi: 10.1109/ICWS.2017.13.

[10] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[11] L. Bonilla Viana, J. Diaz-de-Arcaya, J. Aguirre Usandizaga, and A. Almeida, "AI-driven self-healing across the edge–cloud continuum: A systematic literature review," *Information and Software Technology*, vol. 198, 108232, 2026, doi: 10.1016/j.infsof.2026.108232.